

1. Time functions

When we work on Linux systems, it is a good practice to know how the system deals with dates and times, how we can retrieve these information and how we can use them.

We have already seen how to retrieve the current time with the command `date`. We examine some options of this command.

```
[linux@fidir] date
Fri Apr 25 05:37:42 +07 2025

[linux@fidir] date -u
Thu Apr 24 22:38:37 UTC 2025

[linux@fidir] date +%D
04/25/25

[linux@fidir] date +%T
05:41:06

[linux@fidir] date +%s
1745534529
```

Without any options, `date` returns the current date and time, passing the option `-u`, it returns the UTC time. The `+` sign is used to specify which format or part of the date the command must return: `D` returns the current date in local format, `T` the current time, `s` the timestamp from the Epoch.

In the `/etc` folder, the symbolic link `localtime` points to the file containing the data of the local timezone, which is in the folder `/usr/share/zoneinfo`.

```
[linux@fidir] ls -l /etc/localtime
lrwxrwxrwx 1 root root 35 Mar 26 20:31 /etc/localtime -> /usr/share/zoneinfo/Asia/Phnom_Penh

[linux@fidir] file /usr/share/zoneinfo/Asia/Phnom_Penh
/usr/share/zoneinfo/Asia/Phnom_Penh: timezone data (fat), version 2, no gmt time flags, no std time flags, no leap seconds, 5 transition times, 6 local time types, 21 abbreviation chars
```

The pointed file is a *timezone data* file (binary file).

In addition, in the `/etc` folder, there is the file containing the information of the current timezone.

```
[linux@fidir] ls -l /etc/timezone
-rw-r--r-- 1 root root 16 Apr 17 08:13 timezone

[linux@fidir] cat /etc/timezone
Asia/Phnom_Penh
```

The `/usr/share/zoneinfo` directories contains files for the various time zones available on the system.

1.1 Getting the time: `time`, `gettimeofday` and `settimeofday`

Linux provides many functions for handling date and time. The `time`¹ function returns the time as the number of seconds elapsed since the Epoch.

```
#include <time.h>

time_t time(time_t *_Nullable tloc);

Returns: the number of seconds if success, -1 if error and errno is set
```

If `tloc` is not `NULL`, the returned value is stored in it. The number returned by this calls, represents the number of seconds since the date “1970-01-01 00:00:00 +0000 (UTC)”.

To retrieve and set the time, the following functions² are available.

```
#include <sys/time.h>

int gettimeofday(struct timeval *restrict tv, struct timezone *_Nullable
                 restrict tz);

int settimeofday(const struct timeval *tv, const struct timezone
                 *_Nullable tz);

Returns: 0 if success, -1 if error and errno is set
```

The `timeval` structure is defined in the `sys/time.h` file header as follows:

```
struct timeval {
    time_t tv_sec;           /* seconds */
    suseconds_t tv_usec;     /* microseconds */
};
```

The `timezone` structure is defined as follows:

```
struct timezone {
    int tz_minuteswest;     /* minutes west of Greenwich */
    int tz_dsttime;         /* type of DST correction */
};
```

Since the `tz` argument is obsolete, it is usually specified as `NULL`.

The `gettimeofday.c` program uses `gettimeofday` and `time` to retrieve the current time.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
```

1 See the `time(2)` manual page

2 See the man page `gettimeofday(2)` manual page

```

4 #include <sys/time.h>

5 int main(int argc, char *argv[]) {
6     struct timeval *tv;
7     int curt;

8     if (gettimeofday(tv, NULL) == -1) {
9         perror("gettimeofday");
10        exit(EXIT_FAILURE);
11    }

12    printf("\nCurrent: %d", tv->tv_sec);
13    if ((curt = time(NULL)) == -1) {
14        perror("time");
15    }

16    printf("\nCurrent: %d", curt);
17 }

```

gettimeofday.c

We run the program and then check the result using the `date` command.

```

[linux@fidir] ./gettimeofday
Current: 1745537558
Current: 1745537558

[linux@fidir] date +%s
1745537560

```

`time` and `gettimeofday` have returned the same value. The execution of `date` has returned a similar value (two seconds greater).

1.2 Setting time with `clock_gettime` and `clock_settime`

In addition to the aforementioned functions, the following functions are also available for retrieving and setting the time³.

```

#include <time.h>

int clock_getres(clockid_t clockid, struct timespec *_Nullable res);

int clock_gettime(clockid_t clockid, struct timespec *tp);

int clock_settime(clockid_t clockid, const struct timespec *tp);

Return: 0 if success, -1 if error and errno is set

```

³ See the `clock_getres(2)` manual page

The `clock_gettime` and `clock_settime` functions, retrieve and set the time of the clock specified by the argument `clockid`. The `clockid` argument is of the type `clockid_t`⁴, which is defined in the `sys/types.h`⁵ header file, and represents the clock on which to act.

A clock can be either system-wide, making it accessible to all processes, or process-specific, where it measures time only within a single process.

All the implementations support the system-wide *real-time clock*, which is defined by the `CLOCK_REALTIME` constant, which represents seconds and nanoseconds since the Epoch. The following table lists the clocks supported by the *glibc* and Linux kernel.

<code>CLOCK_REALTIME</code>	System-wide clock that measures real time, to set it it requires appropriate privileges
<code>CLOCK_REALTIME_ALARM</code>	Like <code>CLOCK_REALTIME</code> but not settable
<code>CLOCK_REALTIME_COARSE</code>	Faster and less precise than <code>CLOCK_REALTIME</code> , not settable
<code>CLOCK_TAI</code>	Nonsettable system-wide clock derived by <code>CLOCK_REALTIME</code> but counting leap seconds ⁶
<code>CLOCK_MONOTONIC</code>	Nonsettable system-wide clock representing monotonic time since an unspecified point in the past, on Linux the value is the number of seconds from when the system booted. It does not count time that the system is suspended
<code>CLOCK_MONOTONIC_COARSE</code>	Faster but less precise version of <code>CLOCK_MONOTONIC</code>
<code>CLOCK_MONOTONIC_RAW</code>	Like <code>CLOCK_MONOTONIC</code> but provides access to hardware-based time
<code>CLOCK_BOOTTIME</code>	Nonsettable system-wide clock identical to <code>CLOCK_MONOTONIC</code> but includes the time that the system is suspended
<code>CLOCK_BOOTTIME_ALARM</code>	Like <code>CLOCK_BOOTTIME</code>
<code>CLOCK_PROCESS_CPUTIME_ID</code>	It measures the CPU time consumed by the process, not settable
<code>CLOCK_THREAD_CPUTIME_ID</code>	It measures the CPU time consumed by the thread, not settable

Tab 1. Clocks

These clock IDs are hard-coded in the System V style, and Linux also supports POSIX clock operations on certain character devices known as *dynamic clocks*.

With appropriate macros we can convert open file descriptors into clock IDs⁷ and pass them to `clock_gettime`, `clock_settime`, `clock_adjtime`.

4 `clockid_t` is an arithmetic type, see the `clockid_t(3)` manual page

5 On Debian-based systems, the header file is `/usr/include/x86_64-linux-gnu/bits/types/clockid_t.h`

6 It does not have discontinuities or frequency adjustments caused by inserting leap seconds. TAI is an acronym of International Atomic Time

7 An example of this technique is available in the `clock_getres(2)` manual page

The `clock_getres` function finds the resolution of the specified clock and stores it in the `res` argument.

The resolution of a clock depends on its implementation. Since the clock resolution defines the unit of time measurement, if the value of the `tp` argument passed to the `clock_gettime` function is not a multiple of the `res` value returned by `clock_getres`, it will be truncated to the nearest multiple of `res`.

The `clock_times.c` program, taken from the `clock_gettime(2)` manual page, demonstrates how to use the POSIX functions `clock_gettime` and `clock_getres` with different system clocks available in Linux.

```
1 #define _XOPEN_SOURCE 600
2 #include <stdbool.h>
3 #include <stdint.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <time.h>
7 #define SECS_IN_DAY (24 * 60 * 60)

8 static void displayClock(clockid_t clock, const char *name, bool
                           showRes)
9 {
10     long days;
11     struct timespec ts;

12     if (clock_gettime(clock, &ts) == -1) {
13         perror("clock_gettime");
14         exit(EXIT_FAILURE);
15     }
16     printf("%-15s: %10jd.%03ld (", name, (intmax_t) ts.tv_sec,
            ts.tv_nsec / 1000000);

17     days = ts.tv_sec / SECS_IN_DAY;
18     if (days > 0)
19         printf("%ld days + ", days);

20     printf("%2dh %2dm %2ds", (int) (ts.tv_sec % SECS_IN_DAY) / 3600,
            (int) (ts.tv_sec % 3600) / 60, (int) ts.tv_sec % 60);
21     printf(")\n");

22     if (clock_getres(clock, &ts) == -1) {
23         perror("clock_getres");
24         exit(EXIT_FAILURE);
25     }

26     if (showRes)
27         printf("resolution: %10jd.%09ld\n", (intmax_t) ts.tv_sec,
            ts.tv_nsec);
```

```

28     }

29 int main(int argc, char *argv[])
30 {
31     bool showRes = argc > 1;
32     displayClock(CLOCK_REALTIME, "CLOCK_REALTIME", showRes);

33     #ifdef CLOCK_TAI
34         displayClock(CLOCK_TAI, "CLOCK_TAI", showRes);
35     #endif
36     displayClock(CLOCK_MONOTONIC, "CLOCK_MONOTONIC", showRes);
37     #ifdef CLOCK_BOOTTIME
38         displayClock(CLOCK_BOOTTIME, "CLOCK_BOOTTIME", showRes);
39     #endif
40     exit(EXIT_SUCCESS);
41 }

```

clock_times.c

At line 1, the program defines the feature test macro `_XOPEN_SOURCE 600`, enabling POSIX.1-2001 functionality required for functions such as `clock_gettime` and `clock_getres`. It also defines the constant `SECS_IN_DAY`, representing the number of seconds in one day ($24 * 60 * 60 = 86400$). At lines 8–28, the program defines the function `displayClock`, which receives a clock identifier (`clockid_t`), the clock name as a string, and a boolean flag indicating whether the clock resolution should also be displayed.

At lines 10–11, the function declares a variable `days`, used to store the number of elapsed days, and a `timespec` structure `ts`, which stores the seconds and nanoseconds returned by the clock functions. At lines 12–15, the program calls `clock_gettime` to retrieve the current value of the specified clock. If the call fails, an error message is printed and the program terminates.

At line 16, the function prints the clock name, the elapsed seconds (`tv_sec`), and the milliseconds obtained from the nanoseconds field (`tv_nsec / 1000000`). Next, at lines 17–19, the program calculates the number of complete days contained in the elapsed time and prints the value if it is greater than zero.

At lines 20–21, the remaining seconds are converted into hours, minutes, and seconds using modulo arithmetic, producing a more human-readable representation of the time. At lines 22–25, the function calls `clock_getres` to retrieve the resolution (precision) of the specified clock. If the call fails, an error message is displayed and the program exits. At lines 26–27, if the boolean variable `showRes` is true, the clock resolution is printed in seconds and nanoseconds. In the `main` function, at line 31, the variable `showRes` is initialized. It becomes true if at least one command-line argument is provided (`argc > 1`). At line 32, the program calls `displayClock` with `CLOCK_REALTIME`, representing the system real-time clock.

At lines 33–35, `displayClock` is conditionally invoked with `CLOCK_TAI` if the system supports this clock type. At line 36, the program displays information for `CLOCK_MONOTONIC`, which is a monotonic clock unaffected by manual system time changes. Finally, at lines 37–39, the program conditionally displays information for `CLOCK_BOOTTIME`, a clock that also includes time spent while the system is suspended.

We execute the program

```
[linux@fidir] ./clock_times
```

```
CLOCK_REALTIME : 1745717821.669 (20205 days + 1h 37m 1s)
CLOCK_TAI      : 1745717821.669 (20205 days + 1h 37m 1s)
CLOCK_MONOTONIC : 125659.771 (1 days + 10h 54m 19s)
CLOCK_BOOTTIME : 125659.771 (1 days + 10h 54m 19s)
```

This execution shows the current values returned by four different Linux clocks.

- `CLOCK_REALTIME` and `CLOCK_TAI` return the current calendar time expressed as the number of seconds elapsed since the Unix Epoch (1 January 1970). The output also converts this value into a human-readable format showing days, hours, minutes, and seconds
- `CLOCK_MONOTONIC` returns the time elapsed since the system boot. This clock increases continuously and is not affected by manual system date changes or NTP adjustments
- `CLOCK_BOOTTIME` is similar to `CLOCK_MONOTONIC`, but it also includes the time during which the system was suspended or in sleep mode

In this execution the system has been running for approximately 1 day, 10 hours, 54 minutes, and 19 seconds. Both monotonic clocks return the same value, indicating that the system has not been suspended since boot. The milliseconds displayed after the decimal point correspond to the nanoseconds field converted to milliseconds.

We pass an argument to retrieve the resolution of the clock.

```
[linux@fidir] ./clock_times 1
CLOCK_REALTIME : 1745717909.375 (20205 days + 1h 38m 29s)
resolution: 0.000000001
CLOCK_TAI : 1745717909.375 (20205 days + 1h 38m 29s)
resolution: 0.000000001
CLOCK_MONOTONIC: 125747.477 (1 days + 10h 55m 47s)
resolution: 0.000000001
CLOCK_BOOTTIME : 125747.477 (1 days + 10h 55m 47s)
resolution: 0.000000001
```

When the program is executed with a command-line argument, the variable `showRes` becomes `true`, causing the program to also display the resolution of each clock using `clock_getres`. The output shows that all the clocks have a resolution of `0.000000001`, which corresponds to 1 nanosecond.

This means the kernel is capable of representing time values with nanosecond precision.

`CLOCK_REALTIME` and `CLOCK_TAI` return the current system time expressed as the number of seconds elapsed since the Unix Epoch (1 January 1970 00:00:00 UTC).

The retrieved value is based on UTC time and is independent from the local timezone configured on the system. The timezone only affects how the timestamp is converted into a human-readable date and time representation. As a consequence, the timestamp value is theoretically identical on all systems, differences may occur only if system clocks are not correctly synchronized.

`CLOCK_MONOTONIC` and `CLOCK_BOOTTIME` instead represent the elapsed time since system startup rather than calendar time.

1.3 Clocks on Linux

On Linux systems, the current date, time, timezone, and clock synchronization status can be displayed using the `timedatectl`⁸ command.

```
[linux@fidir] timedatectl
      Local time: Sun 2025-04-27 09:07:48 +07
      Universal time: Sun 2025-04-27 02:07:48 UTC
          RTC time: Sun 2025-04-27 02:07:48
          Time zone: Asia/Phnom_Penh (+07, +0700)
System clock synchronized: yes
      NTP service: active
      RTC in local TZ: no
```

Obviously, the *local time* and the *universal time* (UTC) differ because the local time includes the *timezone offset* configured on the system.

The value returned by `clock_gettime` with `CLOCK_REALTIME` is based on the Unix Epoch (1 January 1970 00:00:00 UTC). Therefore, the returned timestamp is internally represented in UTC and is independent from the *local timezone*.⁹ To demonstrate this behavior we can use the following sequence of commands.

```
[linux@fidir] date +%s
1745720874

[linux@fidir] date -d '1970-01-01 00:00:00' +%s
-25200
[linux@fidir] echo $(( $(date +%s) + -25200 ))
1745695689
[linux@fidir] ./clock_times
CLOCK_REALTIME : 1745720890.867 (20205 days + 2h 28m 10s)
CLOCK_TAI :      1745720890.867 (20205 days + 2h 28m 10s)
CLOCK_MONOTONIC: 2664.448 ( 0h 44m 24s)
CLOCK_BOOTTIME : 2664.448 ( 0h 44m 24s)
```

The first command returns the current timestamp expressed as the number of seconds elapsed since the Unix Epoch. The second command calculates the difference, in seconds, between the Unix Epoch in UTC and the same date interpreted in the local timezone¹⁰. The returned value (25200) corresponds to a timezone offset of $25200 / 60 / 60 = 7$ hours, which means the system timezone is UTC+7.

The third command executes the `clock_times.c` program and displays the values returned by the different clocks. The value returned by `CLOCK_REALTIME` (1745720890.867) is very close to the timestamp produced by `date +%s`, confirming that `CLOCK_REALTIME` internally uses UTC-based Unix time. The small difference of a few seconds is caused simply by the time required to type and execute the commands.

⁸ See the `timedatectl(1)` manual page

⁹ The option `-d` of `date` allows to specify a date and time

¹⁰ As we were in the Epoch date and time in a different timezone on the earth

As a result we understand that Linux internally stores time as UTC, the Unix timestamp is independent from the local timezone, the timezone is only applied when converting timestamps into human-readable local dates and times.

On Linux systems, there are two different types of clocks:

Hardware Clock

Independent hardware device with its own power supplier. It operates when the machine is powered off or even unplugged. Also called “Real time clock” (RTC), BIOS clock, CMOS clock.

System Clock

Is part of the Linux kernel and is driven by a timer interrupt, it exists and has meaning only while Linux is running. It is the number of seconds from the Epoch. This is initialized from the Hardware Clock at boot.

To retrieve information from the *Hardware clock*, we can use the `hwclock`¹¹ utility.

```
[linux@fidir] sudo hwclock
2025-04-27 09:49:11.767010+07:00
```

The command displays:

- the current date
- the current time with fractions of seconds stored by the hardware clock
- the timezone offset

The value shown is retrieved directly from the RTC (Real-Time Clock) located on the motherboard.

1.4 Setting a new time: `tm`

We present a simple example of the use of the `settimeofday` system call. The `setnewtime.c` program first retrieves the current system time and then sets a new date and time value. To define the new date and time, it uses the `tm` structure, which is defined in the `sys/types.h`¹² header file as follows.

```
struct tm
{
    int tm_sec;           /* Seconds. [0-60] (1 leap second) */
    int tm_min;           /* Minutes. [0-59] */
    int tm_hour;          /* Hours. [0-23] */
    int tm_mday;          /* Day. [1-31] */
    int tm_mon;           /* Month. [0-11] */
    int tm_year;          /* Year - 1900. */
    int tm_wday;          /* Day of week. [0-6] */
    int tm_yday;          /* Days in year. [0-365] */
    int tm_isdst;         /* DST. [-1/0/1] */
    # ifdef __USE_MISC
```

¹¹ See the `hwclock(1)` manual page

¹² On Debian-based systems: `/usr/include/x86_64-linux-gnu/bits/types/struct_tm.h`

```
    long int tm_gmtoff;        /* Seconds east of UTC. */
    const char *tm_zone;      /* Timezone abbreviation. */
# else
    long int __tm_gmtoff;      /* Seconds east of UTC. */
    const char *__tm_zone;     /* Timezone abbreviation. */
# endif
};
```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/time.h>
5 #include <time.h>
6 #include <errno.h>

7 int main(int argc, char *argv[]) {
8     struct timeval new_time;
9     time_t t;

10    if (argc != 7) {
11        printf("\nUsage: %s year month day hour min sec", argv[0]);
12        exit(EXIT_FAILURE);
13    }

14    gettimeofday(&new_time, NULL);
15    t = new_time.tv_sec;
16    printf("\nCurrent system time: %s", ctime(&t));
17    struct tm tm_time = {
18        .tm_year = atoi(argv[1]) - 1900,
19        .tm_mon = atoi(argv[2]) - 1,
20        .tm_mday = atoi(argv[3]),
21        .tm_hour = atoi(argv[4]),
22        .tm_min = atoi(argv[5]),
23        .tm_sec = atoi(argv[6]),
24    };

25    time_t new_epoch = mktime(&tm_time);
26    if (new_epoch == -1) {
27        perror("mktime failed");
28        exit(EXIT_FAILURE);
29    }

30    new_time.tv_sec = new_epoch;
31    new_time.tv_usec = 0;

32    if (settimeofday(&new_time, NULL) == -1) {
33        perror("settimeofday");
34        exit(EXIT_FAILURE);
```

```

35     }

36     gettimeofday(&new_time, NULL);
37     t = new_time.tv_sec;
38     printf("New system time: %s", ctime(&t));
39     exit(EXIT_SUCCESS);
40 }

```

setnewtime.c

At line 8, the program declares a `struct timeval` named `new_time`, used to store time values in seconds and microseconds¹³. This structure is required by both `gettimeofday` and `settimeofday`, which provide higher precision than a simple `time_t` value. At line 9, a variable of type `time_t` is declared to store Unix epoch time¹⁴. At lines 14–16, the program retrieves the current system time using `gettimeofday`, which fills the `new_time` structure with the current seconds (`tv_sec`) and microseconds (`tv_usec`). The seconds field is copied into `t` and converted into a human-readable format using `ctime`, allowing the current system time to be displayed¹⁵. At lines 17–24, the program initializes a `struct tm` structure using the date and time values passed on the command line. Each field is converted from string to integer using `atoi`¹⁶. Since the `struct tm`¹⁷ format expects the year as “years since 1900” and months in the range 0–11, the program subtracts 1900 from the year and 1 from the month. At line 25, the filled `struct tm` structure is converted into a Unix epoch timestamp using `mktime`¹⁸. If the conversion fails, the program prints an error message and terminates. At lines 30–31, the computed epoch time is assigned to `new_time.tv_sec`, while the microseconds field is explicitly set to zero. This prepares the structure for use with `settimeofday`. At lines 32–35, the program attempts to update the system clock using `settimeofday`¹⁹. This operation usually requires administrator privileges; otherwise, the call fails with a permission error. Since the system clock is modified directly, the change affects the entire operating system. Finally, at lines 36–38, the program retrieves the updated system time again using `gettimeofday` and prints it in human-readable form with `ctime`, confirming that the new time has been applied successfully.

We execute the program using `sudo`

```
[linux@fidir] sudo ./setnewtime 2025 5 3 17 21 10
```

```
Current system time: Thu Apr 3 17:33:07 2025
```

```
New system time: Sat May 3 17:21:10 2025
```

```
[linux@fidir] date
```

```
Sat May 3 17:21:13 +07 2025
```

There are some things to consider:

¹³ POSIX time format

¹⁴ This is the standard scalar representation of system time used by many C library functions

¹⁵ `ctime` returns a pointer to a statically allocated string that includes a trailing newline, and it is not thread-safe, `ctime_r` would be safer in multithreaded contexts

¹⁶ `atoi` performs no validation, invalid input (non-numeric strings) will silently become 0.

¹⁷ This structure represents *local time*, meaning results depend on the system timezone and daylight saving rules

¹⁸ `mktime` interprets the input as local time and may normalize invalid date values

¹⁹ `settimeofday` is considered obsolete on some systems in favor of `clock_settime`

1. If we run the program without `sudo`, we get "Operation not permitted". This happens because changing the system clock via `settimeofday` is a *privileged operation*
2. The field `tm_year` of the `tm` structure must be expressed as the number of years since 1900. This is part of the historical POSIX/C design of `struct tm`, where: `tm_year = actual_year - 1900`. This is why we subtract 1900 from the user-provided year to correctly map it into the expected internal representation used by `mktime`
3. Months in `tm` are zero-indexed (January = 0, December = 11). This is another legacy design choice in the *C time API*. Therefore `tm_mon = actual_month - 1` is required to align user-friendly input (1–12) with the internal representation expected by `mktime`
4. If NTP (Network Time Protocol) is active on the system, the time may automatically be corrected after running the program. Modern Linux systems often run a time synchronization service (e.g., `systemd-timesyncd`, `ntpd`, or `chronyd`). These services periodically adjust the system clock to match external time servers. As a result, manual changes made by `settimeofday` can be: quickly overwritten, or gradually corrected (slewed back to the correct time, depending on configuration)
5. The program is *not thread-safe*. Several functions used are inherently unsafe in multithreaded environments: `ctime` returns a pointer to a static internal buffer, which can be overwritten by concurrent calls. `mktime` may also rely on shared internal timezone state. Global time-related state (like timezone settings) may be modified implicitly.

1.5 Time conversion functions

The `mktime` function converts a broken-down time structure (`struct tm`) into a calendar time representation (`time_t`), expressed as seconds since the Unix epoch. It ignores the values provided in the `tm_wday` (day of the week) and `tm_yday` (day of the year) fields, since these can be automatically computed from the other fields (year, month, and day).

The `tm_isdst` field indicates whether Daylight Saving Time (DST) is in effect for the given time:

- A positive value means DST is in effect
- A value of 0 means DST is not in effect
- A negative value tells `mktime` to automatically determine whether DST should be applied based on the system's timezone rules

When `tm_isdst` is set to `-1`, the function consults the system's timezone database and adjusts the resulting `time_t` value accordingly if DST applies.

The following functions²⁰ can be used to transform date and time to broken-down time or ASCII.

```
#include <time.h>

char *asctime(const struct tm *tm);
char *asctime_r(const struct tm *restrict tm, char buf[restrict 26]);
```

²⁰ See the `ctime(3)` manual page

```

char *ctime(const time_t *timep);
char *ctime_r(const time_t *restrict timep, char buf[restrict 26]);

struct tm *gmtime(const time_t *timep);
struct tm *gmtime_r(const time_t *restrict timep, struct tm *restrict
                    result);

struct tm *localtime(const time_t *timep);
struct tm *localtime_r(const time_t *restrict timep, struct tm *restrict
                      result);

time_t mktime(struct tm *tm);

Return:
    gmtime and localtime return a pointer to a struct tm, NULL on error
    gmtime_r and localtime_r return the address of the structure pointed to by
        result, NULL on error
    asctime and ctime return a pointer to a string, NULL on error
    asctime_r and ctime_r return a pointer to the string pointed to by buf, NULL
        on error
    mktime returns a calendar time (seconds since Epoch) or -1 if error

```

The functions `ctime`, `gmtime` and `localtime`, take an argument of type `time_t` representing a calendar time, when interpreted as an absolute value, it represent the seconds passed since Epoch. `asctime` and `mktime` take an argument representing broken-down time stored in a structure of type `tm`.

A call to `ctime` is equivalent to call `asctime(localtime(t))` and returns a null-terminated string representing the current time.

The `gmtime` function converts the calendar time `timep` to broken-down representation expressed in UTC.

`gmtime_r` works like `gmtime` but stores the value in an user-supplied structure passed as second argument.

`localtime` converts the calendar time `timep` to broken-down time representation, expressed relative to the user timezone. This function also sets the external variables `tzname`, `timezone` and `daylight`.

`localtime_r` does the same but stores the result in a user-supplied structure passed as second argument.

`asctime` converts the broken-down time value `tm` into a string with the same format as `ctime`.

`asctime_r` does the same but stores the result in an user-supplied structure passed as second argument.

The `mktime` function modifies the fields `tm_wday` and `tm_yday` calculating them based on the values of the other fields. `mktime` also sets the external variables `tzname`, `timezone` and `daylight`.

The variables `tzname`, `timezone` and `daylight` are usually initialized by the function `tzset`²¹. This function is automatically called by the *time conversion functions* that depend on the timezone. The variables are defined as follows.

21 See the `tzset(3)` manual page

```
#include <time.h>

void tzset(void);

extern char *tzname[2];
extern long timezone;
extern int daylight;
```

tzset initializes tzname from the TZ environment variable, if TZ is not defined in the environment the system timezone is used. The system timezone is configured copying or linking a file in the tzfile²² format to the file /etc/localtime. If TZ is in the environment but it has no value then UTC is used. The tzfile format is used to store the timezone data, the files for the several timezones are in the folder /usr/share/zoneinfo.

```
[linux@fidir] ls /usr/share/zoneinfo
Africa      Cuba      GMT+0      Kwajalein  Poland    Zulu
America     EET       GMT-0      Libya      Portugal  iso3166.tab
Antarctica  EST       GMT0       MET         ROC       leap-seconds.list
Arctic      EST5EDT   Greenwich  MST         ROK       leapseconds
Asia        Egypt     HST        MST7MDT     Singapore localtime
Atlantic    Eire      Hongkong   Mexico      Turkey    posixrules
Australia   Etc       Iceland    NZ          UCT       right
Brazil      Europe    Indian     NZ-CHAT     US        tzdata.zi
CET         Factory   Iran       Navajo      UTC       zone.tab
CST6CDT     GB        Israel     PRC         Universal zone1970.tab
Canada      GB-Eire   Jamaica    PST8PDT     W-SU      zonenow.tab
Chile       GMT       Japan      Pacific     WET
```

22 See the tzfile(5) manual page